INTERNAL ARCHITECTURE DOCUMENT

Ishyiga Assistant

End-to-end logic structure

WhatsApp Cloud API → Express backend → Neon Postgres

Groq AI → Client-info API → Personalized system prompt

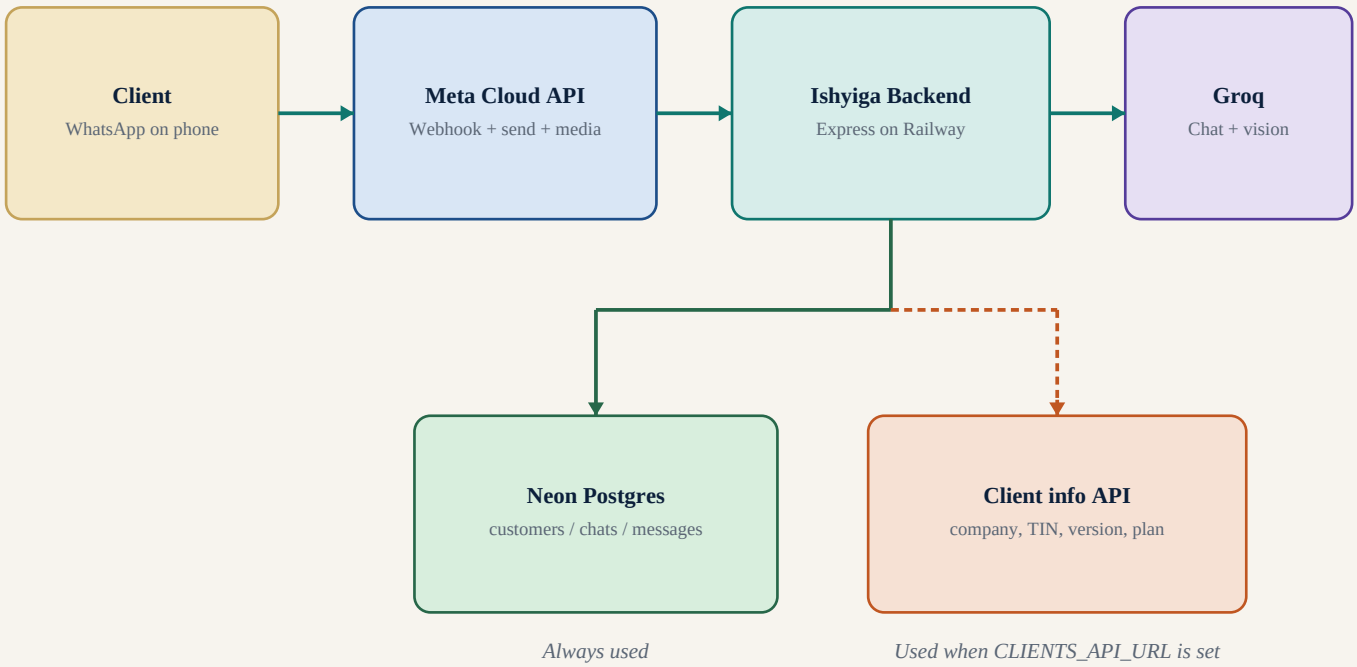
1. Current live message path, from Meta webhook to WhatsApp reply
2. Decision tree: duplicates, screenshots, contact rules, fallbacks
3. Planned / now-wired client API that fills the AI system prompt
4. Data model, components, failure modes, and how to plug the API in

23 August 2026 · Ishyiga Software · Version 1.0

13 pages · Source of truth: backend/src in the ISHYIGA-ASSISTANT repository

Who talks to whom

A client writes to the Ishyiga WhatsApp business number. Meta delivers that event to the Railway backend. The backend stores the conversation, looks up the client record when the client API is configured, asks Groq for a reply, and sends that reply back through the WhatsApp Cloud API.



The client API is optional today. If the URL is empty, missing, slow, or returns 404, the assistant still replies using the base Ishyiga support prompt. When you provide the live URL, the same path starts injecting that client's facts into the system prompt automatically.

Human Meta Backend AI Database Client API

Layers inside the backend

HTTP enters Express. Routes only dispatch. Controllers own the conversation. Services talk to Meta, Groq, Postgres, and the future client API. Models are the only SQL layer.

HTTP edge

app.js · /webhook · /api/health · /api/messages

Controllers

webhookController · healthController · messagesController

Domain services

whatsappService · conversationService · contactRules · clientProfileService · openaiService

Persistence

customer / conversation / message models → Neon Postgres

External AI + Meta

Groq chat + vision · Graph API v23+ send / typing / media

Production host: Railway at /webhook and /api/health. Meta live webhook points at Railway, not ngrok. Local development still uses port 4000 plus a tunnel when needed.

POST /webhook — accept, then work

Meta must receive HTTP 200 quickly. The controller verifies the signature, parses the payload, answers 200, and only then processes text and image events. GET /webhook is the one-time verify handshake.

1. Raw body kept

express.json verify stores req.rawBody

2. HMAC check

X-Hub-Signature-256 vs WHATSAPP_APP_SECRET

3. Parse events

object=whatsapp_business_account, field=messages

4. Drop noise

echo of our own number, stale > 10 min, unsupported types

5. Reply 200

JSON { status: received } — Meta is done

6. processTextEvents

async work after the HTTP response

Filters before a message is treated as work

- Missing from/id, or payload not a WhatsApp business account → reject or ignore.
- from equals the business display number → echo ignored.
- timestamp older than 10 minutes → stale ignored (avoids replay storms).
- Text, image, or image-as-document is kept. Other types become kind=unsupported and never get a reply.

processTextEvents — the live path

Mark read + typing

Graph API typing indicator

Typing starts immediately so the client sees activity while we work.

Persist inbound

customer → open conversation → message

Unique WhatsApp message ids prevent double replies on Meta retries.

Duplicate?

same WhatsApp message id → stop

Load history

prior customer + assistant turns

History excludes the current inbound id so Groq does not see it twice.

Contact rule?

+250788880066 + kimenyi → AIMABLE

Special contact rule wins. Groq and the client API are not called.

Image?

download media → vision model

Unreadable screenshot → fixed sentence, no Groq call.

Client API

lookup by WhatsApp number

Groq generateReply

base prompt + client record + history

Client API failure is fail-open: empty context, still call Groq.

Wait typing window

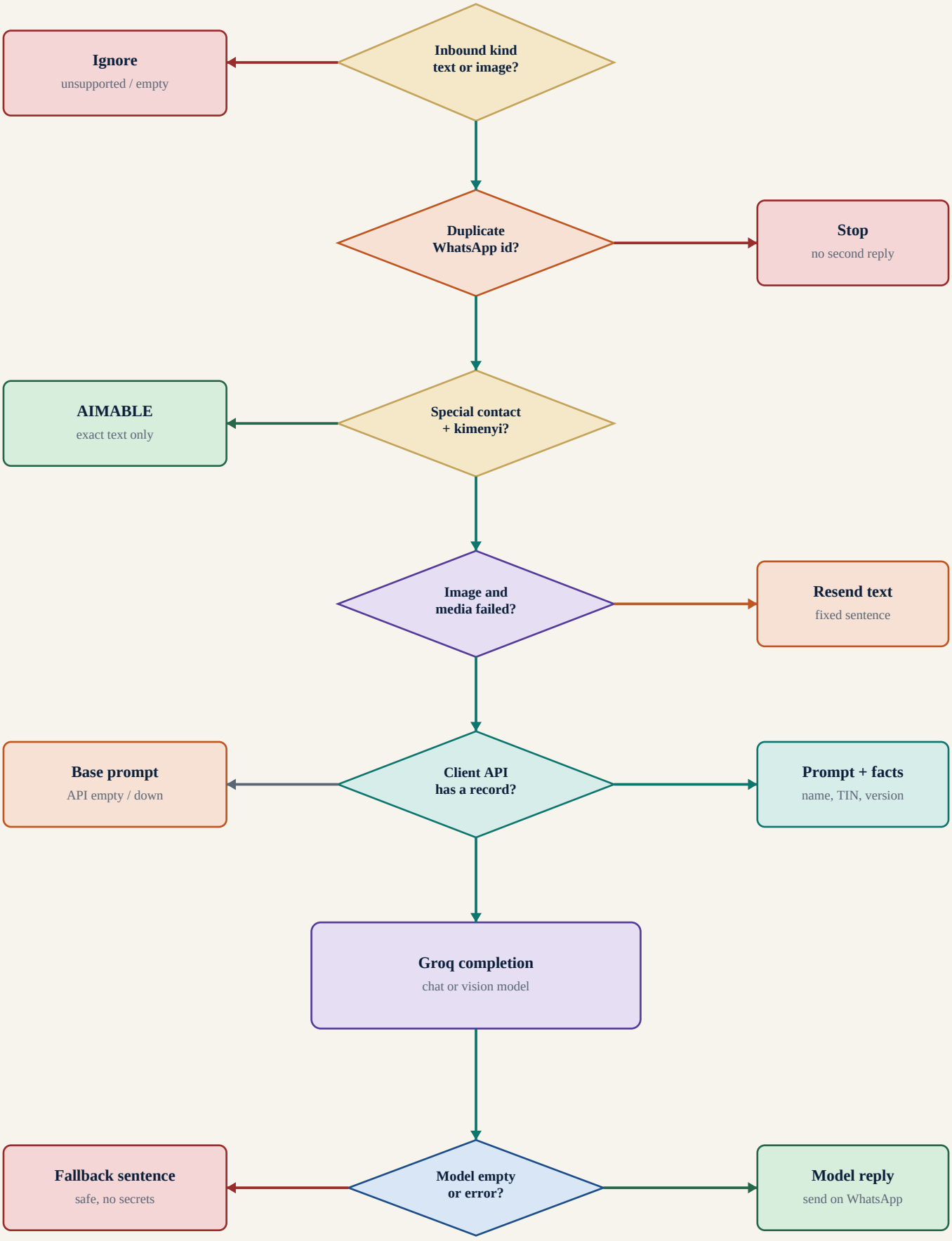
minimum 2 seconds visible

If send fails, the assistant text is not stored as a delivered reply.

Send + persist outbound

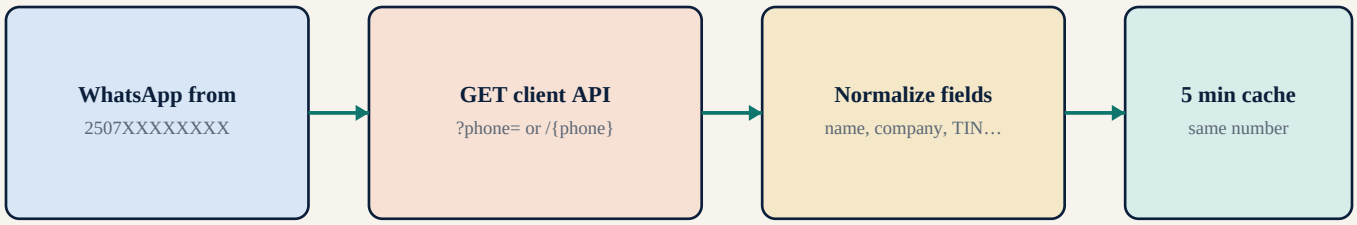
only persist if Meta accepted the send

What reply is sent?



How client facts enter the AI

This is the path that becomes live the moment you give CLIENTS_API_URL. The backend looks up the sender's WhatsApp number, normalizes whatever JSON the API returns, and appends a CURRENT CLIENT RECORD block under the fixed Ishyiga support prompt.



Assembled Groq input

1. SYSTEM = fixed Ishyiga support prompt + optional CURRENT CLIENT RECORD

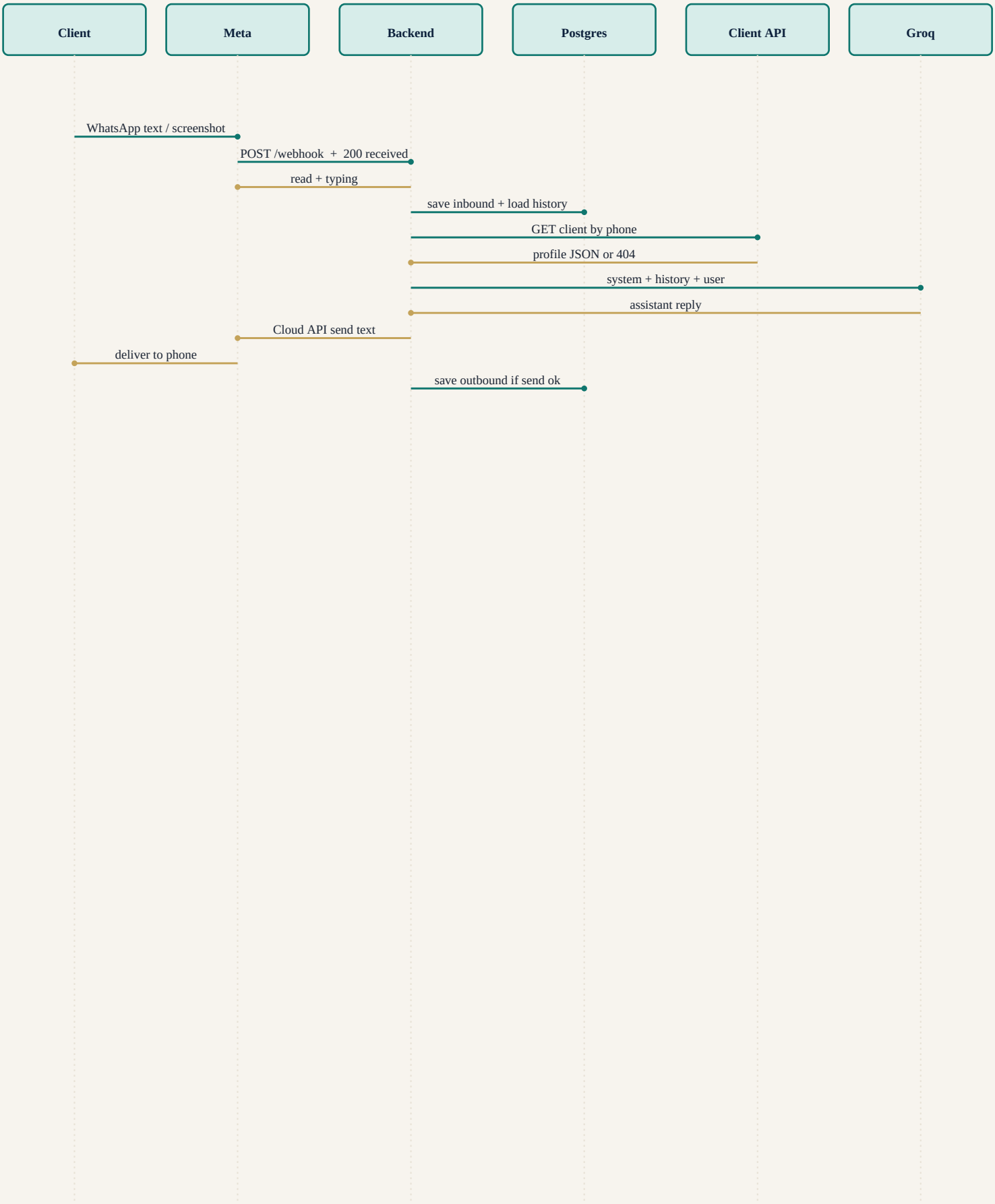
2. HISTORY = previous customer and assistant messages from Postgres

3. USER = this inbound text, plus screenshot bytes when the message is an image

Accepted API shapes (no rewrite when you send the URL)

- GET {CLIENTS_API_URL}?phone=2507… or GET https://host/clients/{phone}
- Optional header: Authorization: Bearer {CLIENTS_API_KEY}
- JSON may be flat, or wrapped as data / client / customer / profile.
- Field aliases are accepted: full_name, business_name, tin_number, app_version, city, package…
- 404 or empty object → no client block. Timeout or 5xx → no client block, Groq still runs.
- Nothing is invented. The model is told to ask the client if a field is missing.

Happy-path timing



What Postgres remembers

The client API is a live lookup. It is not copied into these tables. Postgres only stores the WhatsApp identity and the chat so the next Groq call has memory.

customers

- id UUID PK
- whatsapp_number UNIQUE
- name
- created_at / updated_at

conversations

- id UUID PK
- customer_id FK
- status open | closed
- one open row per customer

messages

- id UUID PK
- conversation_id FK
- whatsapp_message_id UNIQUE
- sender_type customer | assistant
- message + message_type

Runtime cache (not in Postgres)

clientProfileService keeps a 5-minute in-memory cache keyed by digits-only phone. A 404 is also cached so a unknown number does not hit the client API on every follow-up message. Process restart clears the cache. This is safe: the next message simply refetches.

Where each rule lives

routes/webhook.js	GET verify + POST receive
controllers/webhookController.js	Orchestrates one inbound event end to end
services/whatsappService.js	Signature, parse, typing, media, send
services/conversationService.js	Find/create customer + conversation + messages
services/contactRules.js	Hard-coded AIMABLE / kimenyi rule
services/clientProfileService.js	Client API lookup, normalize, prompt block
services/openaiService.js	Base SYSTEM_PROMPT + Groq chat/vision
config/env.js	CLIENTS_API_URL / KEY / TIMEOUT_MS
models/* + migrations	Postgres schema and queries
/api/messages	Local Groq test without WhatsApp
/api/health	DB + Groq + WhatsApp + clientsApiConfigured

The process stays up

If this happens	What the system does
Invalid Meta signature	403, no processing
Malformed payload	400, no processing
Echo / stale / unsupported	Logged, no reply
Duplicate WhatsApp id	Inbound saved once, no second reply
Read/typing fails	Continue; still generate and send
History load fails	Empty history, still reply
Client API down / timeout	Empty client block, still Groq
Client API 404	Unknown client, still Groq
Screenshot download fails	Fixed resend sentence, no Groq
Groq timeout / 429 / auth	Safe fallback sentence
WhatsApp send fails	Reply not persisted as delivered
Special contact rule	Bypasses Groq and client API

When you give the API, this is all that changes

The lookup, prompt assembly, cache, and fail-open behaviour are already in the backend. You do not need a new webhook path. Set three environment variables on Railway (and locally) and restart.

Environment

```
CLIENTS_API_URL=https://your-host/api/clients

CLIENTS_API_URL=https://your-host/api/clients/{phone}

CLIENTS_API_KEY=optional-bearer-token

CLIENTS_API_TIMEOUT_MS=8000
```

Preferred JSON (any extra fields are ignored)

```
{
  "name": "Jean Uwimana",
  "company": "Kigali Mart",
  "tin": "102345678",
  "software": "Ishyiga",
  "version": "4.2.1",
  "plan": "Standard",
  "location": "Kigali",
  "notes": "Prefers Kinyarwanda"
}
```

If your API already uses different names (full_name, business_name, tin_number, app_version), leave it as-is. The adapter maps those aliases. If the contract is completely different (POST body, nested list, custom auth header), send the spec and only the adapter in clientProfileService.js needs a small update.

Health check: GET /api/health → integrations.clientsApiConfigured becomes true once CLIENTS_API_URL is set. The WhatsApp number used for lookup is digits-only, so +250 788 880 066 and 250788880066 are the same client.

Two-layer system prompt

Layer A is always present. It is the Ishyiga Software support constitution: personality, troubleshooting method, RRA rules, security, language, WhatsApp style, and the AIMABLE contact rule. Layer B is only present when the client API returned a usable record.

Layer A — always

- Role: Ishyiga support assistant
- Tone: patient, human, not robotic
- Diagnose before assuming cause
- Network, version, DB, RRA, credentials
- Ask for screenshots when useful
- Never invent status, versions, tickets
- Never request passwords or tokens
- Match the client's language
- Contact rule: kimenyi → AIMABLE

Layer B — when API hits

- Name and company
- TIN if the API sent it
- Installed product / version
- Plan or license
- Location / branch
- Support notes
- Do not invent missing fields
- Do not leak identifiers casually
- Use facts already known

Security boundaries

• Phone numbers are masked in logs (**** last 4).
• Client API key stays in environment variables, never in git.
• Webhook HMAC is checked when WHATSAPP_APP_SECRET is set.
• The model is forbidden from asking for passwords, tokens, or database secrets.
• TIN and email from the client API are for the model's context, not for broadcasting.